



DG INTERNS HUB

Cybersecurity Internship Program

WEEK 2 ASSIGNMENT REPORT

Advanced Networking, Traffic Analysis & Security Concepts

Submitted by: Mah Noor

Report Date: 27 August 2026

Tools Used: Wireshark | Nmap | Cisco Packet Tracer

Table of Contents

1. Introduction and Objective
2. Part 1 – Theory
 - 2.1 Task 1: Ports and Services
 - 2.2 Task 2: Networking Devices (Router, Switch, Firewall)
 - 2.3 Task 3: Network Security Basics (MITM, DoS, Packet Sniffing)
3. Part 2 – Practical Work
 - 3.1 Task 4: Packet Analysis using Wireshark
 - 3.2 Task 5: Network Scanning using Nmap
 - 3.3 Task 6: Packet Tracer LAN Setup
 - 3.4 Task 7: Man-in-the-Middle (MITM) Concept Simulation
4. Tools Used – Summary
5. Learnings and Conclusion

1. Introduction and Objective

This report is written for Week 2 of the DG Interns Hub Cybersecurity Internship. The goal of this week was to move a step ahead from basic networking and get hands-on with real tools. In this task, I studied important networking concepts like ports, routers, switches and firewalls, and then I practiced using tools such as Wireshark, Nmap and Cisco Packet Tracer. I also learned about common network attacks like Man-in-the-Middle (MITM), Denial of Service (DoS) and Packet Sniffing, and how we can prevent them.

All the practical work shown in this report was done by me on my own laptop, inside safe and legal environments. For example, I only scanned scanme.nmap.org, which is a website made by the Nmap team especially for people to practice scanning on. I did not scan or attack any device that I do not own or have permission for.

2. Part 1: Theory

2.1 Task 1: Ports and Services

A port is like a small door number on a computer. When data comes to a computer over the internet, the IP address tells the data which computer to go to, and the port number tells it which application on that computer should receive it. For example, when you open a website, your browser talks to port 80 or port 443 on the web server. Ports are divided into two main types based on protocol: TCP (Transmission Control Protocol), which makes sure data arrives fully and in order, and UDP (User Datagram Protocol), which is faster but does not guarantee delivery. Below is a table of 20 common ports with their protocol, service name and simple usage.

Port No.	Protocol	Service	Usage (in simple words)
20	TCP	FTP (Data)	Transfers files as part of active FTP sessions
21	TCP	FTP (Control)	Sends commands to control file transfers
22	TCP	SSH	Lets admins log in to a remote machine securely
23	TCP	Telnet	Old remote login method, sends data in plain text
25	TCP	SMTP	Sends email between mail servers
53	TCP/UDP	DNS	Turns website names into IP addresses
67	UDP	DHCP (Server)	Gives out IP addresses to devices on a network
68	UDP	DHCP (Client)	Receives an IP address from the DHCP server
69	UDP	TFTP	Simple file transfer, mostly used for device configs
80	TCP	HTTP	Loads normal, unencrypted websites

Port No.	Protocol	Service	Usage (in simple words)
110	TCP	POP3	Downloads email from a mail server to a device
119	TCP	NNTP	Used for newsgroup/discussion services
123	UDP	NTP	Keeps computer clocks in sync over the network
143	TCP	IMAP	Reads email while keeping it stored on the server
161	UDP	SNMP	Lets admins monitor routers, switches and servers
389	TCP	LDAP	Looks up user and directory information
443	TCP	HTTPS	Loads websites with encryption (secure browsing)
445	TCP	SMB	Shares files and printers on Windows networks
3306	TCP	MySQL	Lets applications talk to a MySQL database
3389	TCP	RDP	Lets a user remotely control a Windows desktop

2.2 Task 2: Networking Devices

Router

A router is a device that connects two or more different networks together and decides the best path for data to travel between them. It works at Layer 3 (Network Layer) of the OSI model and uses IP addresses to move data from one network to another. A simple real-world example is the Wi-Fi router at home: it connects your home network to your Internet Service Provider's network, so all your devices can reach the internet.

Switch

A switch connects multiple devices inside the same local network (LAN) and forwards data only to the exact device it is meant for, using MAC addresses. It works at Layer 2 (Data Link Layer). A real-world example is an office network, where a switch connects all computers, printers and other devices in one room or building so they can share files with each other quickly.

Firewall

A firewall is a security device (or software) that checks incoming and outgoing traffic and decides what to allow and what to block, based on a set of rules. It acts like a guard standing at the entrance of a network. A real-world example is a company firewall that blocks unknown or unsafe websites and stops hackers from getting into the internal network.

Router vs Switch – Key Difference

- A router connects different networks together; a switch connects devices inside one single network.

- A router uses IP addresses to send data; a switch uses MAC addresses.
- A router works at Layer 3 of the OSI model; a switch works at Layer 2.
- A router is normally used at the edge of a network (connecting to the internet); a switch is used inside the local network.

2.3 Task 3: Network Security Basics

Network security means protecting a computer network and the data flowing through it from unauthorized access, misuse, or attacks. It includes tools, rules and practices such as firewalls, encryption, strong passwords, and monitoring, all working together to keep data safe and private.

Man-in-the-Middle (MITM) Attack

In a MITM attack, an attacker secretly places themselves between two devices that are communicating, such as a victim and a website or router. The attacker can then read, or even change, the data being sent, without either side realizing it. It works on the idea of intercepting traffic before it reaches its real destination.

Prevention: Always use HTTPS websites, use a VPN on public Wi-Fi, avoid untrusted or open networks, and keep software updated.

Denial of Service (DoS) Attack

In a DoS attack, the attacker sends a huge amount of traffic or requests to a server or network, more than it can handle. This overloads the system so that real, genuine users cannot access the service anymore. If many computers are used together to do this, it is called a DDoS (Distributed Denial of Service) attack.

Prevention: Use firewalls and rate-limiting, use load balancers, and use DDoS protection services offered by cloud providers.

Packet Sniffing

Packet sniffing means capturing and reading the data packets that travel across a network. Tools like Wireshark can be used for this. It is not always bad — network admins use it to fix problems — but attackers can use it to steal usernames, passwords or other private information if the data is not encrypted.

Prevention: Use encrypted connections (HTTPS, VPN, WPA2/WPA3 Wi-Fi), and avoid sending sensitive data over public or unknown networks.

3. Part 2: Practical Work

3.1 Task 4: Packet Analysis using Wireshark

For this task, I opened Wireshark, started a live capture on my Wi-Fi network, then visited a few websites such as google.com, and finally stopped the capture so I could study the traffic. Wireshark shows every packet with its source IP, destination IP, protocol and length, which makes it very useful to understand what is really happening on a network.

The first screenshot below shows the raw, unfiltered capture. I can see different kinds of traffic such as TCP, ICMPv6 (Neighbour Solicitation) and MDNS, along with the source and destination addresses for each packet.

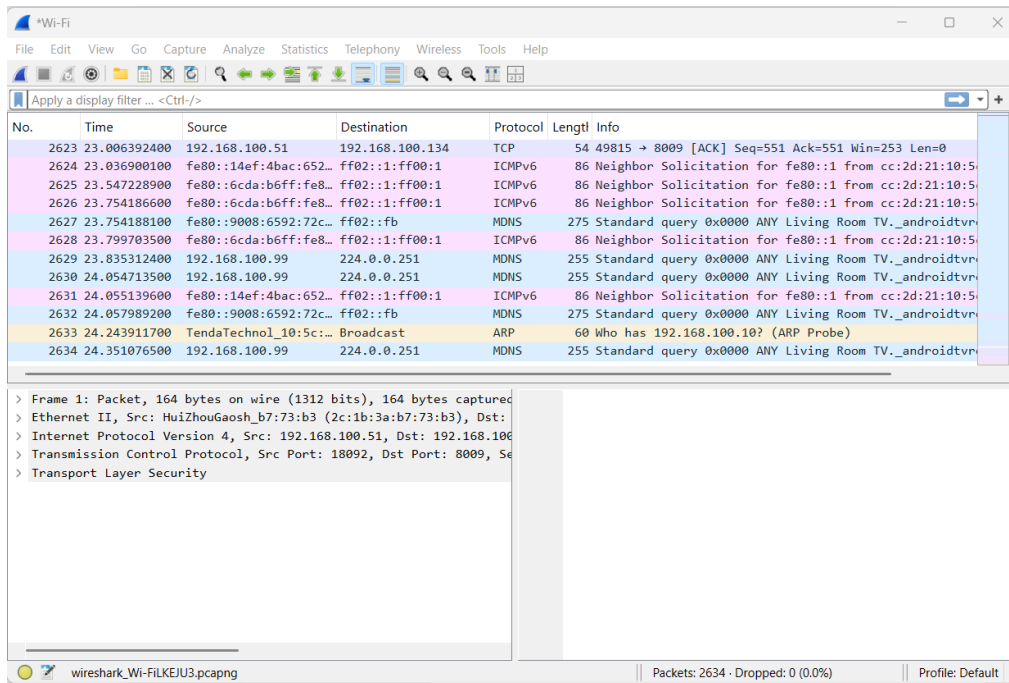


Figure 1: Live packet capture in Wireshark showing mixed traffic (TCP, ICMPv6, MDNS)

Next, I applied the filter "dns" to see only DNS traffic. DNS is the service that turns a website name like www.google.com into an IP address. In the screenshot, my computer (192.168.100.51) sends a DNS query for www.google.com, and the router (192.168.100.1) replies back with the matching IP addresses (A and AAAA records).

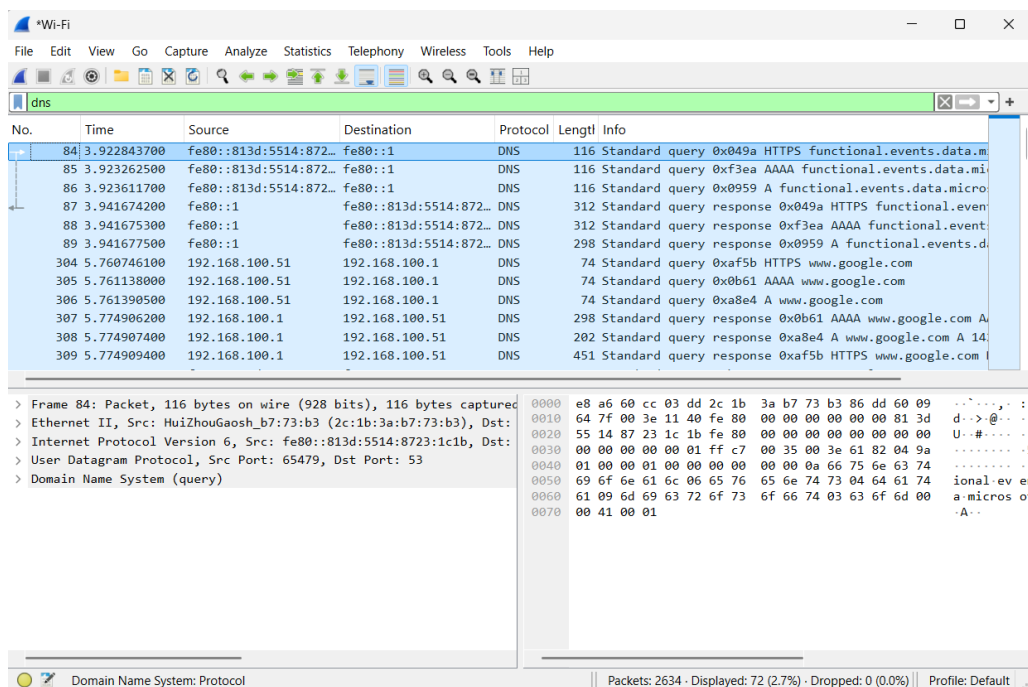


Figure 2: DNS query and response captured for www.google.com

Finally, I applied the filter "tls" to look at encrypted web traffic. TLS is the security layer used by HTTPS. Here I can see "Application Data" packets flowing between my machine and different servers, and one packet even shows a TLS "Client Hello" that contains the server name (SNI) being requested. This confirms that the browsing traffic was encrypted and protected.

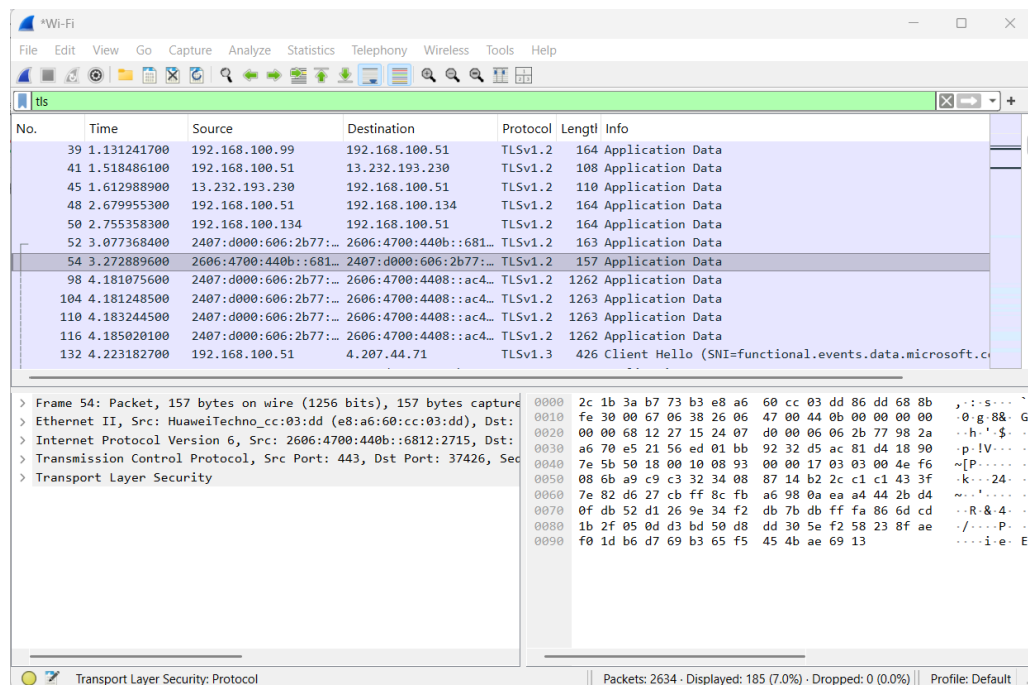


Figure 3: TLS-encrypted traffic (HTTPS) captured between client and server

What I identified from this task

- DNS requests: my PC (192.168.100.51) querying for domains such as www.google.com

- HTTPS/TLS traffic: encrypted Application Data packets and a Client Hello with SNI
- Source and destination IPs: local addresses like 192.168.100.51 / 192.168.100.1, and public server IPs

3.2 Task 5: Network Scanning using Nmap

Before scanning, I first checked that my internet connection and DNS were working properly by pinging 8.8.8.8 and google.com. Both replied successfully with 0% packet loss, which confirmed my network was ready for the scan.

```
(mahnoor@kali)-[~/Desktop/dg]
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=137 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=150 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=119 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=48.6 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3012ms
rtt min/avg/max/mdev = 48.606/113.728/149.640/39.102 ms

(mahnoor@kali)-[~/Desktop/dg]
$ ping -c 4 google.com
PING google.com (142.250.186.14) 56(84) bytes of data:
64 bytes from lcmcta-ak-in-f14.1e100.net (142.250.186.14): icmp_seq=1 ttl=128 time=535 ms
64 bytes from lcmcta-ak-in-f14.1e100.net (142.250.186.14): icmp_seq=2 ttl=128 time=67.9 ms
64 bytes from lcmcta-ak-in-f14.1e100.net (142.250.186.14): icmp_seq=3 ttl=128 time=131 ms
64 bytes from lcmcta-ak-in-f14.1e100.net (142.250.186.14): icmp_seq=4 ttl=128 time=60.3 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 60.262/198.581/535.400/196.374 ms

(mahnoor@kali)-[~/Desktop/dg]
$ getent hosts scanme.nmap.org
```

Figure 4: Connectivity check using ping before running Nmap

For the actual scan, I used scanme.nmap.org, which is the official, legal target provided by the Nmap project for practice. My first command checked the 20 most common ports:

`nmap -Pn -T4 --top-ports 20 scanme.nmap.org`

```
(mahnoor@kali)-[~/Desktop/dg]
$ nmap -Pn -T4 --top-ports 20 scanme.nmap.org
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-26 23:44 +0530
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.37s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f

PORT      STATE      SERVICE
21/tcp    closed     ftp
22/tcp    open       ssh
23/tcp    closed     telnet
25/tcp    filtered   smtp
53/tcp    closed     domain
80/tcp    open       http
110/tcp   closed     pop3
111/tcp   closed     rpcbind
135/tcp   closed     msrpc
139/tcp   closed     netbios-ssn
143/tcp   closed     imap
443/tcp   closed     https
445/tcp   closed     microsoft-ds
993/tcp   closed     imaps
995/tcp   closed     pop3s
1723/tcp  closed     pptp
3306/tcp  closed     mysql
3389/tcp  closed     ms-wbt-server
5900/tcp  closed     vnc
8080/tcp  closed     http-proxy

Nmap done: 1 IP address (1 host up) scanned in 8.56 seconds
```


Figure 5: Nmap scan of the top 20 ports on scanme.nmap.org

Port	State	Service
21/tcp	closed	ftp
22/tcp	open	Ssh
23/tcp	closed	telnet
25/tcp	filtered	Smtp
53/tcp	closed	Domain
80/tcp	open	http
443/tcp	closed	https
3389/tcp	closed	ms-wbt-server

Next, I added the -sV flag, which tells Nmap to also detect the exact software version running behind each open port. This is more detailed than a plain scan:

`nmap -Pn -T4 -sV --top-ports 20 scanme.nmap.org`

```
(mahnoor@kali) - [~/Desktop/dg]
$ nmap -Pn -T4 -sV --top-ports 20 scanme.nmap.org
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-27 00:42 +0530
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.86s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f

PORT      STATE      SERVICE      VERSION
21/tcp    closed    ftp
22/tcp    open      ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
23/tcp    closed    telnet
25/tcp    filtered  smtp
53/tcp    closed    domain
80/tcp    open      http         Apache httpd 2.4.7 ((Ubuntu))
110/tcp   filtered  pop3
111/tcp   closed    rpcbind
135/tcp   filtered  msrpc
139/tcp   closed    netbios-ssn
143/tcp   filtered  imap
443/tcp   filtered  https
445/tcp   closed    microsoft-ds
993/tcp   filtered  imaps
995/tcp   closed    pop3s
1723/tcp  filtered  pptp
3306/tcp  closed    mysql
3389/tcp  closed    ms-wbt-server
5900/tcp  filtered  vnc
8080/tcp  closed    http-proxy
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 29.92 seconds
```

Figure 6: Service and version detection scan (-sV) on scanme.nmap.org

Port	State	Service	Version / Note
22/tcp	open	ssh	OpenSSH 6.6.1p1 (Ubuntu Linux)
80/tcp	open	http	Apache httpd 2.4.7 (Ubuntu)

As an extra step, I also ran a focused scan on just three important ports (22, 80 and 443) with version detection, to practice narrowing down a scan instead of checking every port:

```
nmap -Pn -T4 -sV -p 22,80,443 scanme.nmap.org
```

```
(mahnoor@kali) - [~/Desktop/dg]
$ nmap -Pn -T4 -sV -p 22,80,443 scanme.nmap.org
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-27 00:44 +0530
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.41s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f

PORT      STATE      SERVICE  VERSION
22/tcp    open      ssh      OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open      http     Apache httpd 2.4.7 ((Ubuntu))
443/tcp   filtered  https
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.39 seconds
```

Figure 7: Focused Nmap scan on ports 22, 80 and 443

What I identified from this task

- Open ports on scanme.nmap.org: 22 (SSH) and 80 (HTTP)
- Services and versions: OpenSSH 6.6.1p1 on Ubuntu, and Apache httpd 2.4.7 on Ubuntu
- Port 443 (HTTPS) was closed/filtered, meaning the server does not serve encrypted web traffic on that host

3.3 Task 6: Packet Tracer LAN Setup

For this task, I used Cisco Packet Tracer to build a very small local area network (LAN) with two computers connected through one switch. This is the most basic building block of any office or home network.

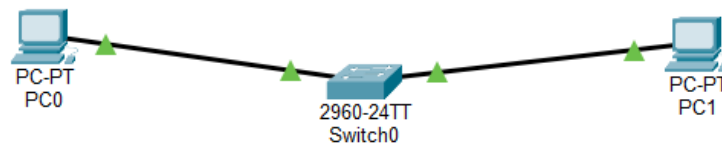


Figure 8: LAN topology – PC0 and PC1 connected through Switch0

I gave each PC a static IP address from the same subnet (192.168.1.0/24) so that they could talk to each other directly through the switch, without needing a router.

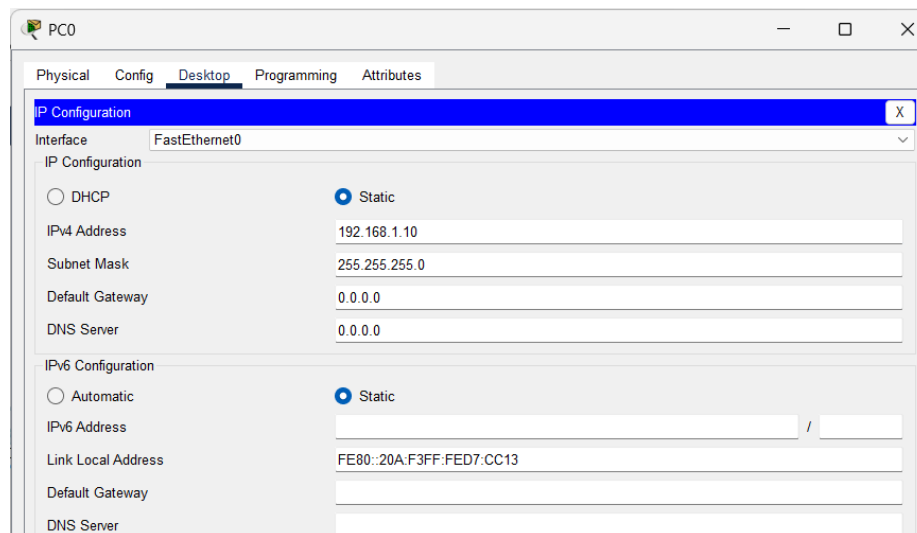


Figure 9: IP configuration of PC0 (192.168.1.10)

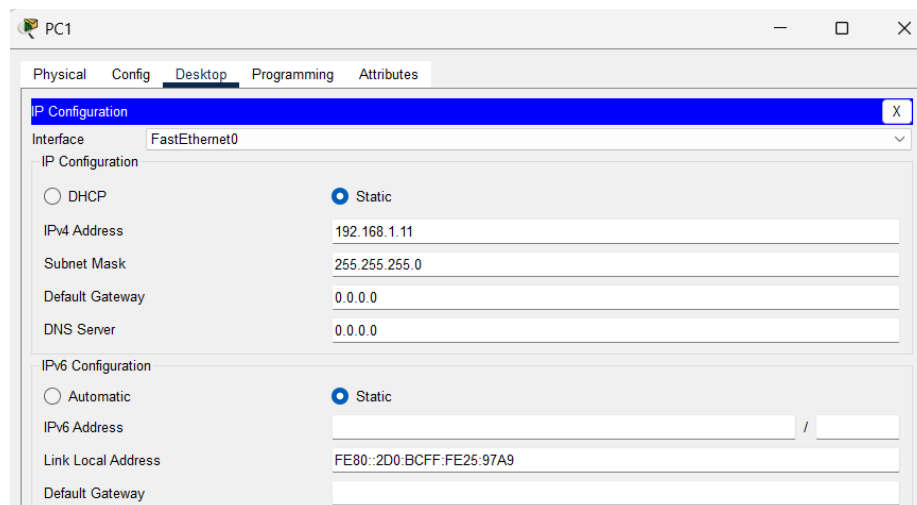


Figure 10: IP configuration of PC1 (192.168.1.11)

After the IP addresses were set, I tested the connection by pinging PC1 from PC0's command prompt. The ping was successful, which proved that both computers were correctly connected and configured on the same LAN.

What I learned from this task

- How to add end devices and a switch in Packet Tracer and cable them together
- How to assign static IP addresses and a subnet mask to a device
- How the ping command is used to test basic connectivity between two devices

3.4 Task 7: Man-in-the-Middle (MITM) Concept Simulation

This task was only a safe, conceptual simulation to understand how a MITM attack looks from a network point of view — no real attack tool was used. I built a small network with a Victim PC, an Attacker PC, a Switch and a Router, so that both PCs sit on the same LAN and the router represents the gateway to the outside world (for example, the internet).

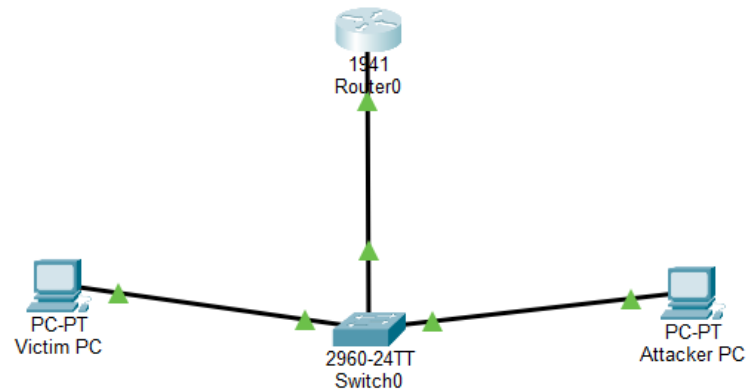


Figure 11: MITM lab topology – Victim PC and Attacker PC connected via Switch to Router0

I assigned static IP addresses to each device so the topology matches a real small office network: the Victim PC, the Attacker PC, and the Router acting as the default gateway.

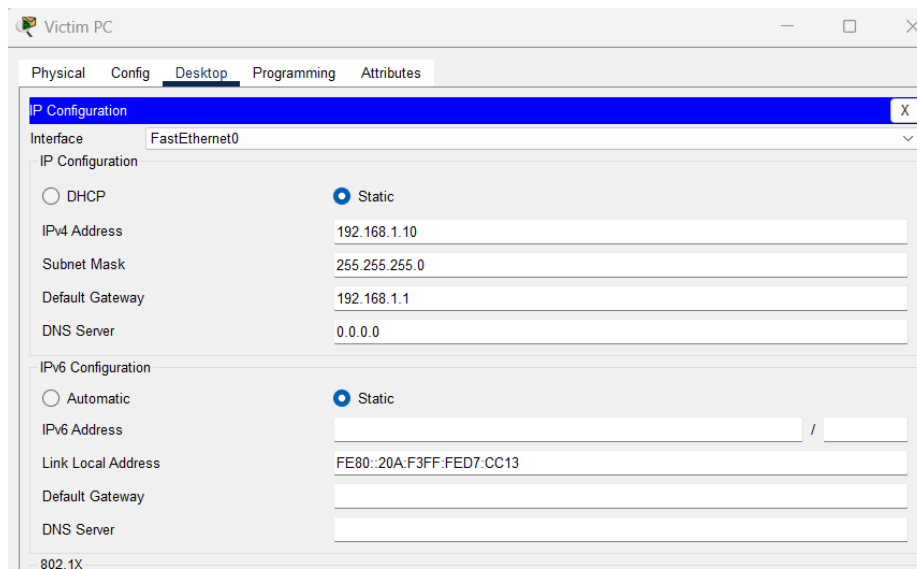


Figure 12: Victim PC IP configuration (192.168.1.10, gateway 192.168.1.1)

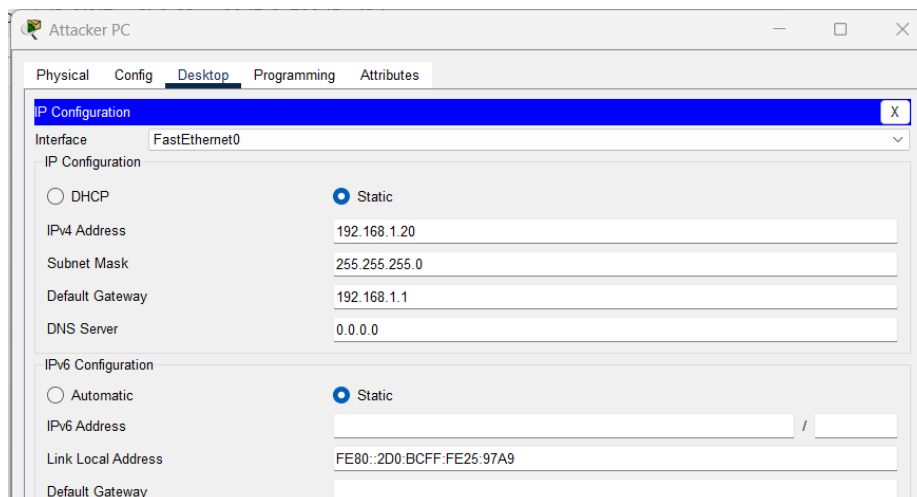


Figure 13: Attacker PC IP configuration (192.168.1.20, gateway 192.168.1.1)

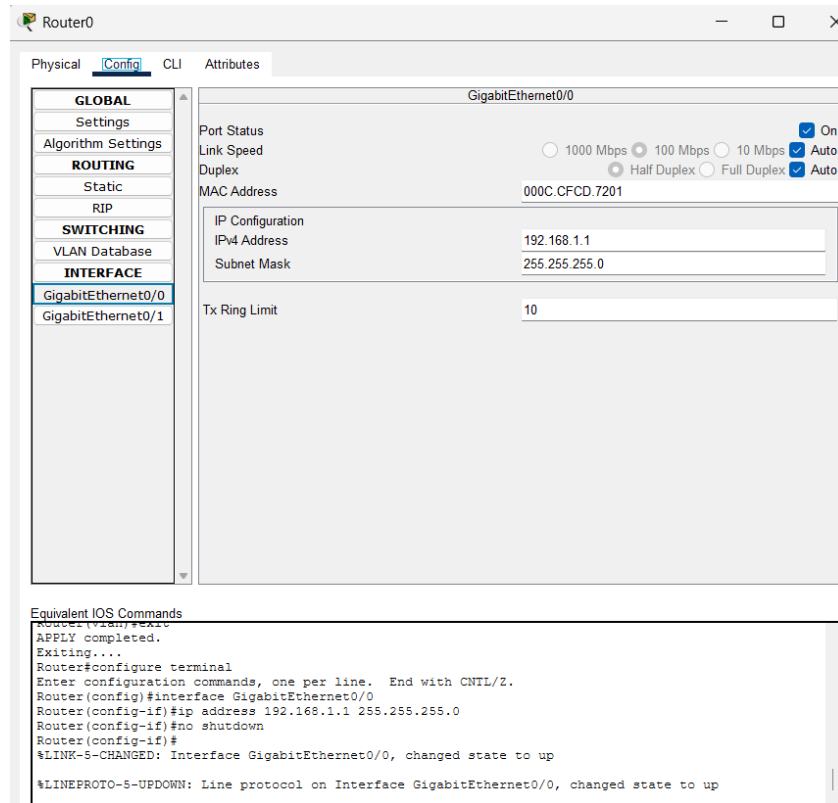


Figure 14: Router0 configuration on interface GigabitEthernet0/0 (192.168.1.1)

Once the addressing was in place, I tested basic reachability. First a quick ping check confirmed replies were coming back correctly on the network:

Then I ran a full ping test from the Victim PC to the Router (gateway) and observed the traffic from the Attacker PC's side as well. Both showed successful replies with 0% packet loss, confirming that the Victim, Attacker and Router could all communicate normally over the shared switch — this is exactly the kind of shared network an attacker would need to be positioned on to attempt a MITM attack in real life.

```

Victim PC
Approximate round trip times in milli-seconds:
  Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>clear
Invalid Command.

C:\>exit

C:\>ping 192.168.1.1

Pinging 192.168.1.1 with 32 bytes of data:

Reply from 192.168.1.1: bytes=32 time<1ms TTL=255
Reply from 192.168.1.1: bytes=32 time<1ms TTL=255
Reply from 192.168.1.1: bytes=32 time<1ms TTL=255
Reply from 192.168.1.1: bytes=32 time<1ms TTL=255

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
      Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
C:\>

Attacker PC
Physical  Config  Desktop  Programming  Attributes

Command Prompt

Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.20

Pinging 192.168.1.20 with 32 bytes of data:

Reply from 192.168.1.20: bytes=32 time=17ms TTL=128
Reply from 192.168.1.20: bytes=32 time=2ms TTL=128
Reply from 192.168.1.20: bytes=32 time=9ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
      Minimum = 0ms, Maximum = 17ms, Average = 7ms

C:\>
  
```

Figure 16: Ping results from Victim PC and Attacker PC confirming connectivity on the shared LAN

What is MITM (in simple words)

Man-in-the-Middle means an attacker gets a position in the middle of two devices that are talking to each other, such as a victim's computer and the router. Because normal traffic on a switch can sometimes be intercepted (for example using ARP spoofing in a real attack), the attacker can quietly read or alter the data passing between the victim and the internet, without the victim knowing anything is wrong.

How the attacker sits in between

In this simulation, the Attacker PC, Victim PC and Router are all connected to the same switch, which means they all share the same local network. In a real attack, the attacker would trick the switch and the victim's computer into sending traffic through the attacker's machine first, before it goes to the router. My lab only shows the network layout and normal connectivity, not the actual interception technique, to keep the exercise safe.

Real-world example

A very common real-world example is public Wi-Fi, such as in a cafe or airport. An attacker connected to the same Wi-Fi can try to position themselves between other users and the Wi-Fi router, and capture unencrypted data such as login details.

Risks of MITM

- Data theft: private messages, files or personal information can be read by the attacker

- Password capture: login credentials can be stolen if the connection is not encrypted
- Session hijacking: an attacker could take over an already logged-in session

Prevention

- Use HTTPS websites, which encrypt the data between the browser and the server
- Use a VPN, especially on public or unknown Wi-Fi networks
- Avoid using open, unsecured Wi-Fi for sensitive activities like online banking
- Keep operating systems and browsers updated with the latest security patches

4. Tools Used – Summary

Wireshark

A free packet-capturing and analysis tool. I used it to capture live traffic on my network and study DNS requests and encrypted (TLS/HTTPS) traffic.

Nmap

A free network scanning tool. I used it to check open ports and detect running services and their versions on the safe practice target scanme.nmap.org.

Cisco Packet Tracer

A free network simulation tool by Cisco. I used it to build small LAN topologies, assign IP addresses, and test connectivity with the ping command, including a safe conceptual MITM topology.

5. Learnings and Conclusion

This week's task helped me understand networking concepts in a much more practical way compared to just reading theory. By using Wireshark, I learned how real traffic looks on the wire and how DNS and HTTPS actually work behind the scenes. By using Nmap, I learned how scanning is done responsibly on a target that allows it, and how open ports and service versions can reveal information about a server. By using Cisco Packet Tracer, I got hands-on practice with building small networks, assigning IP addresses, and testing connectivity.

I also learned the basic concepts behind common attacks such as MITM, DoS and packet sniffing, along with simple, practical ways to prevent them, such as using HTTPS, VPNs and firewalls. Overall, Week 2 gave me a stronger foundation in both networking and basic cybersecurity concepts, and I am looking forward to building on these skills in the coming weeks.